

Rust Ownership, Closures, Lifetimes, and Smart Pointers

Merged sheet based on ECM2433 Rust ownership, closures/iterators, lifetimes, and smart-pointer revision topics. It keeps related borrowing and ownership rules together instead of splitting them into several small sheets.

What You Need To Know

1. Closures capture their environment

A closure is an inline function-like value. It can borrow, mutably borrow, or move variables from the surrounding scope. The capture style affects whether the closure implements `Fn`, `FnMut`, or `FnOnce`.

```
let threshold = 10;
let bigger = |n: &i32| *n > threshold; // borrows threshold

let mut count = 0;
let mut inc = || {
    count += 1; // mutable borrow
};
inc();
```

2. Iterators are lazy until consumed

Adapters such as `map` and `filter` return another iterator. Nothing is fully produced until a consuming method such as `collect`, `sum`, or `count` runs.

```
let values = vec![1, 2, 3, 4];
let squares = values
    .iter()
    .filter(|n| **n % 2 == 0)
    .map(|n| *n * *n)
    .collect::<Vec<_>>();
```

3. Lifetimes describe borrowed relationships

Lifetime annotations do not extend how long data lives. They describe which borrowed output is tied to which borrowed input, so Rust can reject dangling references.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}
```

4. Box and Rc

`Box<T>` stores one owned value on the heap. It is useful for large values and recursive types. `Rc<T>` gives shared ownership in one thread by keeping a reference count. Cloning an `Rc` clones the pointer and increments the count; it does not clone the inner data.

```
use std::rc::Rc;

let shared = Rc::new(String::from("notes"));
let a = Rc::clone(&shared);
let b = Rc::clone(&shared);
println!("{}", Rc::strong_count(&shared)); // 3
```

Practice Questions

Question 1 - explain the capture:

```
let factor = 3;
let scale = |n: i32| n * factor;
println!("{}", scale(4));
```

1 Does this closure borrow or move `factor`? Explain briefly.

Question 2 - fix the mutable closure:

```
let mut count = 0;
let inc = || {
    count += 1;
};
inc();
```

2 Fix the binding so the closure can be called.

Question 3 - complete the iterator:

```
let values = vec![1, 2, 3, 4, 5, 6];
```

```

let squares = values
    .iter()
    .filter(|n| **n % 2 == 0)
    .map(|n| /* fill this */)
    .collect::<Vec<_>>();

```

- 3 Fill the map step so the result is [4, 16, 36].
- 4 Why does map alone not build a vector?
- 5 Write the signature for `longest`, which returns one of two borrowed string slices.

Question 6 - identify the lifetime error:

```

fn bad() -> &str {
    let text = String::from("hello");
    &text
}

```

- 6 Explain why this cannot compile and give one normal fix.

Question 7 - read from a Box:

```

fn read_boxed(n: &Box<i32>) -> i32 {
    /* return the inner integer */
}

```

- 7 Complete the function body.

Question 8 - fix the recursive type:

```

enum List {
    Cons(i32, List),
    Nil,
}

```

- 8 Fix this enum so Rust knows its size.

Question 9 - trace the count:

```

use std::rc::Rc;

let value = Rc::new(String::from("x"));
let a = Rc::clone(&value);
{
    let b = Rc::clone(&value);
    println!("{}", Rc::strong_count(&value));
}
println!("{}", Rc::strong_count(&value));

```

- 9 What two numbers are printed, and why?
- 10 Why is `Rc<T>` not the right shared-ownership tool for multiple threads?

Mark Scheme

Attempt all questions before checking.

- 1 It borrows `factor` immutably because `i32` can be copied and the closure only reads it. A move closure would capture its own copy.
- 2 Use `let mut inc = || { count += 1; };`. Calling a closure that mutates captured state needs a mutable closure binding.
- 3 Use `*n * *n`. In the map step, `n` is `&i32`.
- 4 `map` is lazy and returns another iterator. Use a consumer such as `collect::<Vec<_>>()` to build a vector.
- 5 `fn longest<'a>(a: &'a str, b: &'a str) -> &'a str`
- 6 It returns a reference to `text`, but `text` is dropped when the function ends. Return an owned `String` or return a reference to input data instead.
- 7 `**n`. One dereference goes from `&Box<i32>` to `Box<i32>`, the next reaches the `i32`.
- 8 Use indirection: `enum List { Cons(i32, Box<List>), Nil }`.
- 9 It prints 3 inside the block and 2 after the block. Dropping `b` decreases the strong count.
- 10 `Rc<T>` updates its count non-atomically and is not thread-safe. Use `Arc<T>` for shared ownership across threads.